Deep Dive — Every Design Decision Explained

Chunks · Embeddings · Cosine Similarity · ChromaDB · LangGraph · Nodes · FastAPI
Why each file exists · Why each choice was made · How it all connects

PART 1	The Data Pipeline	rag/ingest.py
PART 2	Embeddings & Cosine	rag/retriever.py
PART 3	ChromaDB — Deep Dive	backend/chroma_db/
PART 4	The File Architecture	all backend files
PART 5	AgentState	agent/state.py
PART 6	Every Node — Deep Dive	agent/nodes/*.py (7 files)
PART 7	graph.py — The Wiring	agent/graph.py
PART 8	FastAPI & Streaming	api/main.py
PART 9	Design Decisions	all files

PART 1 — The Data Pipeline

■ backend/rag/ingest.py

Before a single user can chat, the system must read 6 PDF books, cut them into pieces, convert each piece into numbers, and store everything in a searchable database. This happens once by running **rag/ingest.py**. It never runs again unless the books change.

Step 1 — Defining the books

```
BOOKS = {
    "Feeling Good":             "books/feeling_good.pdf",
    "Attached":                 "books/attached.pdf",
    "The Body Keeps the Score": "books/body_keeps_score.pdf",
    "Games People Play":        "books/games_people_play.pdf",
    "Thinking Fast and Slow":   "books/thinking_fast_and_slow.pdf",
    "Nonviolent Communication": "books/nonviolent_communication.pdf",
}
```

A Python dictionary maps a clean display name to the PDF file path. The display name becomes the book's identity — stored as metadata on every chunk and shown in the sidebar 'Used' badges.

> **Why this design choice:** The display name is stored instead of the file path because file paths are internal details. The display name is what gets shown to users and cited in responses.

Step 2 — Loading the embedding model

```
embeddings = HuggingFaceEmbeddings(
    model_name="sentence-transformers/all-MiniLM-L6-v2",
    model_kwargs={"device": "cpu"},
    encode_kwargs={"normalize_embeddings": True},
)
```

- **sentence-transformers/all-MiniLM-L6-v2** — converts sentences into 384-number vectors. Runs locally on CPU, no API, no cost, no rate limits.
- **normalize_embeddings: True** — scales every vector to magnitude=1. Makes cosine similarity about direction (meaning) only — not affected by text length.

> **Why this design choice:** Local model instead of OpenAI/Google embeddings avoids API costs and rate limits during ingest. Processing 7,267 chunks through a paid embedding API would cost money and could be throttled.

Step 3 — Splitting books into chunks

```
splitter = RecursiveCharacterTextSplitter(chunk_size=800, chunk_overlap=120)
```

```
for book_name, pdf_path in BOOKS.items():
    loader = PyPDFLoader(pdf_path)
    pages  = loader.load()
    chunks = splitter.split_documents(pages)
    for chunk in chunks:
        chunk.metadata["source"] = book_name   # tag with book name
    all_chunks.extend(chunks)
```

Why split into chunks at all?

- **LLMs have context limits** — you cannot feed a 400-page book into the AI. You need to find and send only the relevant pages.
- **Embedding precision** — a vector for an entire chapter represents everything and nothing. A vector for one paragraph has a focused, precise meaning.
- **Retrieval accuracy** — smaller chunks produce more accurate similarity scores.

How RecursiveCharacterTextSplitter works

Tries to cut at natural boundaries in priority order:

```
1st: paragraph breaks (\n\n)  ← cleanest split
2nd: line breaks (\n)
3rd: sentence endings (. )
4th: word boundaries ( )
5th: hard cut at 800 chars    ← last resort only
```

Most chunks are 400–800 characters. chunk_size=800 is a maximum, not a fixed size.

The overlap

```
Chunk 1: chars   0 → 800
Chunk 2: chars 680 → 1480  ← starts 120 chars before chunk 1 ended
Chunk 3: chars 1360 → 2160
```

chunk_overlap=120 ensures a sentence at the boundary of two chunks appears fully in at least one of them — nothing is cut in half.

Step 4 — Storing in ChromaDB

```
Chroma.from_documents(all_chunks, embeddings, persist_directory='./chroma_db')
```

For every chunk: calls HuggingFace to convert text → 384-number vector → stores raw text + vector + metadata in ChromaDB → saves to disk at **backend/chroma_db/**.

Book	Chunks
Feeling Good	1,864
Attached	622
The Body Keeps the Score	1,765

Games People Play	512
Thinking Fast and Slow	1,834
Nonviolent Communication	670
TOTAL	7,267

PART 2 — Embeddings & Cosine Similarity

■ backend/rag/retriever.py

What is an embedding?

An embedding converts the MEANING of text into a point in 384-dimensional mathematical space. Texts with similar meanings end up at similar points.

```
"I hate my boss"           → [0.23, -0.15, 0.87, 0.44, ... ]  384 numbers
"conflict with my manager" → [0.21, -0.14, 0.85, 0.46, ... ]  very close!
"I love pizza"             → [0.89,  0.67, -0.23, 0.11, ...]  very different
```

The 384 numbers together encode semantic meaning. Individual numbers cannot be read — it is the relationship between all 384 that captures meaning.

Key insight: The SAME model must be used at ingest time AND query time. Model A for chunks + Model B for queries = different mathematical spaces = meaningless comparisons = random results.

What is cosine similarity?

Cosine similarity measures the angle between two 384-dim vectors. It only cares about direction — not magnitude. normalize_embeddings=True ensures magnitude is irrelevant.

```
"I feel lonely" vs "emotional disconnection" → angle ≈ 8°  → score near 0 → SIMILAR
"I feel lonely" vs "how to bake bread"       → angle ≈ 85° → score near 2 → DIFFERENT
```

- ChromaDB compares the query vector against all 7,267 stored chunk vectors
- Chunks sorted by smallest distance (most similar) to largest
- Top 8 returned as the most relevant passages
- Matches MEANING not keywords — 'I feel empty' finds 'emotional numbness' without shared words

Why this design choice: normalize_embeddings=True ensures a 800-char chunk and a 200-char chunk are compared fairly. Without normalization, longer text = larger magnitude = higher scores regardless of actual relevance.

The retriever singleton

```
# rag/retriever.py
_embeddings  = HuggingFaceEmbeddings(model_name="...", ...)  # loaded ONCE at import
_vectorstore = Chroma(persist_directory="./chroma_db", ...)  # opened ONCE at import
_retriever   = _vectorstore.as_retriever(
    search_type="similarity",
    search_kwargs={"k": 8},              # return top 8 chunks per query
)

def build_retriever():
```

```
    return _retriever                    # returns the pre-built singleton
```

All three objects are created once when Python imports this file — not once per request. Loading the HuggingFace model takes ~2 seconds. If reloaded per request, the app would be unusably slow.

> **Why this design choice:** build_retriever() is a function not a module-level variable so callers import cleanly: build_retriever().invoke(query). The implementation details stay hidden inside retriever.py.

PART 3 — ChromaDB: Deep Dive

```
■   backend/chroma_db/ (database folder on disk)
```

ChromaDB is the vector database that stores all 7,267 book chunks. It lives entirely on your local disk and requires no external service, no API key, and no network connection to search.

What is a vector database?

A regular SQL database searches by exact keyword matching:

```
SELECT * FROM chunks WHERE text LIKE '%conflict%'
```

This misses 'disagreement', 'argument', 'tension' — same concept, different words. A vector database searches by **semantic similarity** — meaning, not exact words:

```
"I fight with my partner" → finds "couple disagreement", "romantic conflict"
                            even without those exact words
```

What ChromaDB physically stores

For every chunk, ChromaDB stores 4 things:

```
# What's inside chroma_db/ for one chunk:
{
    "id":        "abc123-uuid...",         # unique identifier (auto-generated)
    "document":  "Attachment theory...",  # the raw text of the chunk
    "embedding": [0.23, -0.15, 0.87, ...],# 384-number vector (the meaning)
    "metadata":  {
        "source": "Attached",             # book name (shown in sidebar)
        "page":   42                      # page number (shown in citations)
    }
}
```

- **id** — a unique UUID. Used internally for deletion and updates.
- **document** — the raw text. Returned to the app after search so the LLM can read it.
- **embedding** — 384 floats. Used for cosine similarity. Never returned to the app — only used internally.
- **metadata** — source and page. Source powers 'Used' badges. Page powers citations in responses.

Where it lives on disk

```
backend/chroma_db/
■■■ chroma.sqlite3                  ← raw documents + metadata stored here
■■■ 89b354d7-1710-43cd-..../        ← collection folder
    ■■■ data_level0.bin             ← HNSW index (the search structure)
    ■■■ header.bin
    ■■■ length.bin
```

```
    ■■■ link_lists.bin
```

chroma.sqlite3 stores the raw documents and metadata in SQL format. The binary files store the HNSW index — the mathematical structure that makes search fast without comparing every vector.

> **Why this design choice:** Everything is local files. No server process, no network. ChromaDB loads from disk once at startup and stays in memory. Zero network latency per search, no account needed, no rate limits.

How HNSW makes search fast

HNSW = Hierarchical Navigable Small World. Without it, searching would compare against all 7,267 vectors one by one — slow but exact. HNSW builds a layered graph:

```
WITHOUT HNSW (brute force):
Query → compare against chunk 1 → chunk 2 → ... → chunk 7,267
Time: 7,267 comparisons every search

WITH HNSW:
Query → enter graph at top layer → jump to approximate neighborhood
      → refine in lower layers   → find nearest neighbors
Time: ~log(7,267) ≈ 13 comparisons  (approximate but 99%+ accurate)
```

- HNSW is approximate — does not guarantee mathematically closest chunks, but finds the same results as brute force 99%+ of the time.
- The index is built once during ingest and loaded from disk at startup. All searches are in-memory.
- At 7,267 chunks even brute force is fast. HNSW becomes essential at millions of chunks.

How a search works step by step

```
Query: "My manager keeps dismissing my ideas in meetings"

Step 1: HuggingFace embeds query → [0.23, -0.15, 0.87, ...]  (384 numbers)

Step 2: ChromaDB enters HNSW index → navigates graph → candidate region

Step 3: Computes cosine distance:
        distance = 1 - cosine_similarity(query_vector, chunk_vector)
        Lower distance = more similar = more relevant

Step 4: Returns top 8 chunks sorted by distance:
        [NVC, p.42]    distance=1.20  ← most similar
        [Feeling Good] distance=1.24
        ...
        [TBKTS, p.335] distance=1.34  ← least similar of the 8
```

The three ways ChromaDB is used in this project

Where	How used	Purpose
rag/ingest.py	Chroma.from_documents(chunks, embeddings, ...)	Write all 7,267 chunks to disk (runs once)
rag/retriever.py	Chroma(...).as_retriever(k=8)	Open DB at startup, build retriever for search
api/main.py	_vectorstore.get() / similarity_search_with_score()	Debug endpoints — inspect stored chunks

The debug endpoints in api/main.py

```
GET /db/stats              → chunk count per book (verify ingest worked)
GET /db/search?q=query     → raw cosine search with distance scores (debug retrieval)
GET /db/browse?book=name   → scroll through chunks of one book (inspect quality)
```

> **Key insight:** These debug endpoints revealed the ≡ encoding problem. Browsing NVC chunks with /db/browse showed private-use area characters (U+E062, U+E09D) from the old OceanofPDF compilation — invisible in normal output but visible when inspecting raw chunk text.

When to rebuild ChromaDB

- A PDF is replaced with a better quality version — old chunks have encoding artifacts
- A new book is added to the knowledge base
- chunk_size or chunk_overlap is changed
- The chroma_db/ folder is deleted or corrupted

Rebuild: delete chroma_db/, run **python rag/ingest.py** from backend/, restart the server.

PART 4 — Why the Files Are Structured This Way

■ backend/ (all files)

```
backend/
■■■ rag/
■   ■■■ ingest.py          ← PART 1: PDF → chunks → ChromaDB (runs once)
■   ■■■ retriever.py       ← PART 2: query → cosine search → chunks
■
■■■ agent/
■   ■■■ state.py           ← PART 5: AgentState — the shared memory
■   ■■■ graph.py           ← PART 7: wiring — which node connects to which
■   ■■■ nodes/
■       ■■■ guard.py       ← PART 6 Node 1: off-topic detection
■       ■■■ retriever.py   ← PART 6 Node 2: search + filtering
■       ■■■ evaluator.py   ← PART 6 Node 3: chunk quality gate
■       ■■■ query_builder.py ← PART 6 Node 4: query rewriter
■       ■■■ psychologist.py  ← PART 6 Node 5: response generator
■       ■■■ action_planner.py← PART 6 Node 6: structured action steps
■       ■■■ follow_up.py   ← PART 6 Node 7: closing question
■
■■■ api/
■   ■■■ main.py            ← PART 8: FastAPI web server + SSE streaming
■
■■■ llm_factory.py         ← shared: creates Groq LLM for all nodes
■■■ start.py               ← loads .env and starts uvicorn server
```

Why is each node in its own file?

Each node has a single, completely different responsibility. Imagine all 7 nodes in one file:

```
# BAD — all in one file
def guard_run(state): ...          # 44 lines
def retriever_run(state): ...      # 55 lines
def evaluator_run(state): ...      # 48 lines
def query_builder_run(state): ... # 30 lines
def psychologist_run(state): ...   # 59 lines
def action_planner_run(state): .. # 44 lines
def follow_up_run(state): ...      # 45 lines
# Total: 325+ lines — impossible to navigate
```

- **Single responsibility** — guard.py does one thing: check if message is on-topic. Zero knowledge of what comes after.
- **Easy to find** — psychologist response wrong → open psychologist.py immediately. No searching through 325 lines.
- **Easy to change independently** — rewrite the evaluator prompt without touching any other file.
- **Easy to test** — each run(state) takes a dict, returns a dict. Test in isolation with fake state.

Why this design choice: Separation of concerns: each file has one reason to change. Guard logic changes → only guard.py changes. Psychologist prompt changes → only psychologist.py. No file ever changes for reasons unrelated to its own job.

Why is graph.py separate from the node files?

graph.py contains NO logic — only wiring. It doesn't know HOW guard works, only THAT guard exists and where it routes to.

Key insight: graph.py is the blueprint. Node files are the workers. Blueprint and workers change for different reasons — redesign the pipeline without touching node logic, improve a node without touching the pipeline.

Why state.py is its own file

AgentState is used by every node AND graph.py. If it lived inside guard.py, then evaluator.py would import from guard.py — a confusing dependency between unrelated nodes. state.py is the shared contract everyone imports from without depending on each other.

Why llm_factory.py is its own file

Five nodes create a Groq LLM. Without llm_factory.py, each repeats the same 4-line configuration. If the model name changes (Llama 4 released), change one line in llm_factory.py — all 5 nodes update automatically.

PART 5 — AgentState: The Shared Memory

■ backend/agent/state.py

AgentState is the whiteboard every node reads from and writes to. Nodes never call each other directly — they only communicate through state.

```
class AgentState(TypedDict):
    messages:            List[dict]          # full conversation history
    current_query:       str                 # what to search (may be rewritten)
    retrieved_chunks:    List[dict]          # book passages found
    retrieval_attempts:  int                 # retry counter, max 3
    is_enough:           bool                # evaluator verdict
    retry_reason:        Optional[str]       # why evaluator said insufficient
    active_frameworks:   List[str]           # books CITED in the response
    action_plan:         Optional[List[str]] # structured action steps
    final_response:      Optional[str]       # the AI's answer
    should_loop:         bool                # loop back for another turn?
    turn_count:          int                 # conversation turn number
    is_off_topic:        bool                # guard verdict
    follow_up_question:  Optional[str]       # closing question
```

Field	Written by	Read by	Why it exists
messages	API (main.py)	guard, psychologist, follow_up	Full conversation for multi-turn context
current_query	guard, query_builder	retriever node	Search query — starts as user message, rewritten up to 3×
retrieved_chunks	retriever node	evaluator, psychologist, planner	The 8 book passages — core data of the pipeline
retrieval_attempts	retriever node	evaluator, graph router	Safety counter — prevents infinite retry loop
is_enough	evaluator	graph router	True = proceed, False = retry
retry_reason	evaluator	query_builder	Tells query_builder exactly what failed
active_frameworks	psychologist	frontend sidebar	Only books cited in response — not all retrieved
action_plan	action_planner	frontend, follow_up	Structured steps shown to user
final_response	guard / psychologist	frontend	The answer text
is_off_topic	guard	graph router	Triggers early exit — skips retrieval
follow_up_question	follow_up	frontend	Closing question shown below response

Why this design choice: TypedDict lets Python's type checker catch errors at edit time — mistyped field names warn you immediately. It also documents the entire data model in 18 lines.

PART 6 — Every Node, Every Line Explained

■ backend/agent/nodes/ (7 separate files)

Node 1: guard.py — The Gatekeeper

backend/agent/nodes/guard.py

```
llm = make_llm(temperature=0)   # deterministic — same input always same output

prompt = ChatPromptTemplate.from_template("""
You are a filter for a psychology-focused assistant.
RELEVANT: conflicts with boss, partner, family; personal struggles.
IRRELEVANT: sports, weather, news, politics, cooking, coding.
User message: {user_message}
Return exactly one word: relevant or off_topic
""")

def run(state: AgentState) -> AgentState:
    user_message = state["messages"][-1]["content"]  # latest message
    state["current_query"]      = user_message        # set initial search query
    state["retrieval_attempts"] = 0                   # reset retry counter

    response = (prompt | llm).invoke({"user_message": user_message})
    raw = response.content.strip().lower()

    if "off_topic" in raw:
        state["is_off_topic"]   = True
        state["final_response"] = _OFF_TOPIC_MESSAGE
        state["action_plan"]    = []
        state["active_frameworks"] = []
    else:
        state["is_off_topic"] = False
    return state
```

- **temperature=0** — classification must be consistent. Same message → same label every time.
- **state['messages'][-1]** — index -1 = last item in the list = the latest user message.
- **current_query = user_message** — sets initial search query. query_builder may later overwrite this.
- **retrieval_attempts = 0** — resets at every new message — prevents retry counter carrying over between turns.
- **(prompt | llm).invoke()** — LangChain pipe: fill template → send to Groq → get response.
- **'off_topic' in raw** — uses 'in' not '==' because LLM might add punctuation. Safer.

Why this design choice: Guard saves 4+ LLM calls per off-topic question. Without it, 'what's the weather?' runs full retrieval, fails evaluation 3 times, and produces a confused response — wasting time and tokens.

Node 2: retriever.py — The Searcher

backend/agent/nodes/retriever.py

```
_retriever = build_retriever()   # singleton — loaded once at startup

_KNOWN_BOOKS = {"Feeling Good", "Attached", "The Body Keeps the Score",
                "Games People Play", "Thinking Fast and Slow",
                "Nonviolent Communication"}

_FOREIGN_MARKERS = [
    "The Gifts of Imperfection", "The 7 Habits", "Crucial Conversations",
    "Daring Greatly", "The Empathy Factor", "Radical Acceptance", ...
]

def _is_clean(text: str) -> bool:
    for marker in _FOREIGN_MARKERS:
        if marker and marker in text: return False   # foreign ref found
    return True

def run(state: AgentState) -> AgentState:
    results = _retriever.invoke(state["current_query"])   # cosine search → 8 docs

    chunks = []
    for doc in results:
        source = doc.metadata.get("source", "Unknown")
        if source not in _KNOWN_BOOKS: continue           # skip unknown source
        if not _is_clean(doc.page_content): continue      # skip foreign ref
        chunks.append({
            "text":        doc.page_content,
            "source_book": source,
            "page":        doc.metadata.get("page", 0),
        })

    state["retrieved_chunks"]   = chunks
    state["retrieval_attempts"] += 1
    return state
```

- **build_retriever()** — returns pre-built singleton from rag/retriever.py. Already in memory.
- **_KNOWN_BOOKS filter** — protects against chunks with unexpected source metadata.
- **_is_clean() + _FOREIGN_MARKERS** — some OceanofPDF PDFs are compilations. A chunk labeled 'NVC' might contain 'The Gifts of Imperfection (p.23)'. This drops those chunks before the psychologist sees them.
- **retrieval_attempts += 1** — when this hits 3, evaluator is forced to accept whatever was found.

> **Why this design choice:** Without _FOREIGN_MARKERS filter, the psychologist reads '[The Gifts of Imperfection, p.23]' inside chunk body text and cites it as a real source — hallucinating a book not in the knowledge base.

Node 3: evaluator.py — The Quality Gate

backend/agent/nodes/evaluator.py

```
llm = make_llm(temperature=0)

prompt = ChatPromptTemplate.from_template("""
User situation: {user_message}
Retrieved excerpts: {chunks}

1. Do at least 2 excerpts contain a real, actionable framework directly
   relevant to THIS situation — not just vague emotional content?
2. Would a psychologist actually use these for this person?
3. Are chunks about the right topic? (manager → workplace, not dates/daydreams)

Be strict. Generic content is NOT sufficient.
Answer: SUFFICIENT or INSUFFICIENT (first line)
One sentence explaining why (second line).
""")

def run(state: AgentState) -> AgentState:
    chunks_text = "\n\n".join([f"[{c['source_book']}] {c['text']}"
                               for c in state["retrieved_chunks"]])
    response = (prompt | llm).invoke({
        "user_message": state["messages"][-1]["content"],
        "chunks":       chunks_text,
    })
    lines = response.content.strip().split("\n")
    state["is_enough"]    = "SUFFICIENT" in lines[0].upper()
    state["retry_reason"] = lines[1].strip() if len(lines) > 1 else ""
    return state
```

- **All chunks formatted together** — evaluator sees user message + all chunks labeled with source book.
- **Strict criteria** — requires at least 2 chunks with named frameworks. Vague emotional content explicitly rejected.
- **'SUFFICIENT' in lines[0].upper()** — checks first line only. 'in' not '==' because LLM may add punctuation.
- **retry_reason saved** — passed to query_builder so it knows exactly what failed and can write a targeted fix.

> **Why this design choice:** The evaluator is what makes this system agentic. Without it, the system blindly answers with whatever chunks were found — even if they were completely irrelevant.

Node 4: query_builder.py — The Query Fixer

`backend/agent/nodes/query_builder.py`

```
llm = make_llm(temperature=0.3)   # slightly creative for better rephrasing

prompt = ChatPromptTemplate.from_template("""
You tried to help with: {user_message}
You searched for: {previous_query}
The chunks were insufficient because: {retry_reason}
```

```
Write a better, more specific search query.
Think: psychological concepts, named frameworks, interpersonal dynamics.
Return only the query, nothing else.
""")

def run(state: AgentState) -> AgentState:
    new_query = (prompt | llm).invoke({
        "user_message":   state["messages"][-1]["content"],
        "previous_query": state["current_query"],
        "retry_reason":   state["retry_reason"],
    })
    state["current_query"] = new_query.content.strip()
    return state
```

```
Example:
  User:      "I feel invisible at work"
  Previous:  "I feel invisible at work"
  Reason:    "general communication chunks, not workplace recognition"
  New query: "being overlooked ignored workplace recognition acknowledgment"
```

- **temperature=0.3** — needs creativity to rephrase differently, not so random it becomes irrelevant.
- **Three inputs** — original message (goal), previous query (what failed), retry_reason (why). Gives LLM everything to write a targeted fix.
- **Only current_query updated** — state['messages'] (original user text) is never changed. Psychologist still sees what the user actually wrote.

Node 5: psychologist.py — The Response Generator

backend/agent/nodes/psychologist.py

```
llm = make_llm(temperature=0.7)   # warm, natural language

_SYSTEM = """...
SITUATION RULE — CRITICAL:
Read user's message to identify who it's about.
Look for: "manager", "boss", "partner", "wife", "colleague", "family", etc.
Use exactly that relationship. "manager" → "your manager". NEVER say "partner".

CITATION RULE — CRITICAL:
Each excerpt starts with [Book Name, p.X].
Cite as: Book Name (p.X). Only cite from the label — not from inside body text.
Only cite a book if you actually used its content.
Allowed: Feeling Good, Attached, The Body Keeps the Score,
         Games People Play, Thinking Fast and Slow, Nonviolent Communication.

Book excerpts: {context}"""

def run(state: AgentState) -> AgentState:
    context = "\n\n".join([
        f"[{c['source_book']}, p.{c['page']}]\n{c['text']}"
        for c in state["retrieved_chunks"]
    ])
```

```
    msgs = [SystemMessage(content=_SYSTEM.format(context=context))]
    for msg in state["messages"][:-1]:           # rebuild conversation history
        if msg["role"] == "user":   msgs.append(HumanMessage(content=msg["content"]))
        elif msg["role"] in ("assistant","ai"): msgs.append(AIMessage(...))
    msgs.append(HumanMessage(content=state["messages"][-1]["content"]))

    response = llm.invoke(msgs)
    state["final_response"] = response.content

    # Only mark book as Used if name appears in the response text
    all_books = list(set([c["source_book"] for c in state["retrieved_chunks"]]))
    state["active_frameworks"] = [b for b in all_books if b in response.content]
    return state
```

- **temperature=0.7** — highest in pipeline. Produces warm, varied, human-sounding responses.
- **context formatted with [Book, p.X] labels** — without labels, LLM has no way to know which book a chunk came from.
- **Full conversation history rebuilt** — essential for meaningful follow-up conversations across multiple turns.
- **active_frameworks filtered by response text** — only books cited in the response are shown as 'Used' in the sidebar.
- **Situation inference in prompt** — infers context from user's message. Saves one LLM call vs a dedicated classifier node.

Node 6: action_planner.py — The Structured Planner

backend/agent/nodes/action_planner.py

```
llm = make_llm(temperature=0.5)

class ActionPlan(BaseModel):
    steps:          List[str]   # specific, behavioural steps
    framework_used: str         # e.g. "NVC — Observation vs Evaluation"
    book_source:    str         # e.g. "Nonviolent Communication"
    time_horizon:   str         # e.g. "This week"

structured_llm = llm.with_structured_output(ActionPlan)

def run(state: AgentState) -> AgentState:
    context = "\n".join([c["source_book"] + ": " + c["text"][:200]
                         for c in state["retrieved_chunks"]])
    try:
        result = (prompt | structured_llm).invoke({
            "response":     state["final_response"],
            "context":      context,
            "user_message": state["messages"][-1]["content"],
        })
        state["action_plan"] = result.steps
    except Exception:
        state["action_plan"] = []   # fallback — never crash the pipeline
```

```
    return state
```

- **with_structured_output(ActionPlan)** — uses Groq's tool-calling API to enforce schema. Response is ALWAYS a valid ActionPlan object, never free text.
- **Why not PydanticOutputParser?** — Old approach asked LLM to return raw JSON text. Groq returned prose and the parser crashed with JSONDecodeError. with_structured_output() enforces structure at the API level.
- **context[:200] per chunk** — only first 200 chars per chunk. Full response already has key insights. Full chunks again would waste tokens.
- **try/except with empty fallback** — pipeline never crashes. Shows empty plan instead of 500 error.

Node 7: follow_up.py — The Conversation Deepener

backend/agent/nodes/follow_up.py

```
llm = make_llm(temperature=0.6)

_SYSTEM = """You just gave advice and an action plan.
Read the conversation history carefully — do NOT repeat questions asked before.
Ask ONE short follow-up question that either:
- Digs deeper into something not yet covered
- Checks if the action plan feels realistic for this person
- Uncovers something that would change your advice
One question only. Warm, conversational tone."""

def run(state: AgentState) -> AgentState:
    # rebuild full conversation history for context
    response = llm.invoke(msgs)
    state["follow_up_question"] = response.content
    state["turn_count"]  += 1
    state["should_loop"]  = False    # ends the graph after this turn
    return state
```

- **Full history passed** — prevents repeating questions from earlier turns.
- **should_loop = False** — ends the graph. If True, graph.py routes back to psychologist for another full turn.
- **turn_count += 1** — tracks conversation depth. Could trigger strategy changes after N turns.
- **temperature=0.6** — warm enough to sound natural, controlled enough to stay on topic.

PART 7 — graph.py: The Wiring

■ **backend/agent/graph.py**

---

graph.py contains NO logic of its own. It only describes which node connects to which, and under what conditions. It is the blueprint; nodes are the workers.

```
from langgraph.graph import StateGraph, END
from agent.nodes import retriever, evaluator, query_builder,
                        psychologist, action_planner, follow_up, guard

# ■■ Router functions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■■
def route_after_guard(state):
    return END if state.get("is_off_topic") else "retriever"

def route_after_evaluation(state):
    if state["is_enough"]:               return "psychologist"
    if state["retrieval_attempts"] >= 3: return "psychologist"  # force after 3
    return "query_builder"

def route_after_followup(state):
    return "psychologist" if state["should_loop"] else END

# ■■ Graph construction ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
■■■■■
def build_graph():
    graph = StateGraph(AgentState)

    graph.add_node("guard",          guard.run)
    graph.add_node("retriever",      retriever.run)
    graph.add_node("evaluator",      evaluator.run)
    graph.add_node("query_builder",  query_builder.run)
    graph.add_node("psychologist",   psychologist.run)
    graph.add_node("action_planner", action_planner.run)
    graph.add_node("follow_up",      follow_up.run)

    graph.set_entry_point("guard")

    graph.add_edge("retriever",     "evaluator")
    graph.add_edge("query_builder", "retriever")    # ← THE RETRY LOOP
    graph.add_edge("psychologist",  "action_planner")
    graph.add_edge("action_planner","follow_up")

    graph.add_conditional_edges("guard",     route_after_guard,
        {"retriever": "retriever", END: END})
    graph.add_conditional_edges("evaluator", route_after_evaluation,
        {"psychologist":"psychologist", "query_builder":"query_builder"})
    graph.add_conditional_edges("follow_up", route_after_followup,
        {"psychologist":"psychologist", END: END})

    return graph.compile()
```

- **add_node('guard', guard.run)** — registers name 'guard' and links it to guard.run. When graph reaches 'guard', it calls guard.run(state).
- **add_edge(A, B)** — fixed connection. After A always go to B. No decision.
- **add_conditional_edges(A, router, {map})** — after A, call router(state) and use its return value to pick next node.
- **add_edge('query_builder','retriever')** — this single line creates the entire retry loop.
- **graph.compile()** — validates graph (no missing nodes, no dead ends) and optimizes. Runs once at startup.

> **Key insight:** The graph is a DAG with one cycle — the retry loop. LangGraph supports cycles. Plain LangChain (LCEL) cannot express loops cleanly — you'd write a manual while loop, losing streaming, state management, and observability.

PART 8 — FastAPI & Real-Time Streaming

■ backend/api/main.py

```
app   = FastAPI(title="MindMate Assistant API")
graph = build_graph()    # compile graph ONCE at startup

app.add_middleware(CORSMiddleware,
    allow_origins=["http://localhost:3000", "http://localhost:5173"], ...)

@app.post("/chat")
async def chat(req: ChatRequest):
    initial_state = {
        "messages":           [m.dict() for m in req.messages],
        "current_query":      req.messages[-1].content,
        "retrieved_chunks":   [],    "retrieval_attempts": 0,
        "is_enough":          False, "retry_reason":       None,
        "active_frameworks":  [],    "action_plan":        None,
        "final_response":     None,  "should_loop":        False,
        "turn_count":         0,     "is_off_topic":       False,
        "follow_up_question": None,
    }

    async def stream():
        async for chunk in graph.astream(initial_state):
            for node_name, state_update in chunk.items():
                safe = {k: v for k, v in state_update.items()
                        if isinstance(v,(str,int,float,bool,list,dict,type(None)))}
                yield f"data: {json.dumps({'node':node_name,'output':safe})}\n\n"
        yield 'data: {"node": "__end__"}\n\n'

    return StreamingResponse(stream(), media_type="text/event-stream")
```

- **graph = build_graph() at module level** — compiled once. Every request reuses the compiled graph.
- **CORSMiddleware** — browsers block requests from React (port 5173) to backend (port 8000). This middleware allows it.
- **initial_state** — fresh AgentState for every request. All fields reset. Conversation history from frontend passed in.
- **graph.astream()** — runs graph asynchronously, yielding one result per node as each finishes.
- **safe dict filter** — keeps only JSON-serializable types. LangChain objects inside state would crash json.dumps.
- **yield f'data: ...'** — Server-Sent Events format. Frontend's EventSource reads these as they arrive in real time.
- **StreamingResponse** — keeps HTTP connection open. Without it, user sees blank screen for 5+ seconds then everything at once.

PART 9 — Every Design Decision Answered

■ all files (cross-cutting decisions)

Why not put all nodes in one file?

Each node has a single responsibility. A 325-line single file is impossible to navigate. guard.py is 44 lines — self-contained and readable in 2 minutes. Separation makes each node independently testable and changeable.

Why is graph.py separate from node files?

graph.py is the blueprint. Node files are workers. Blueprint and workers change for completely different reasons — redesign the pipeline without touching node logic, improve a node without touching the pipeline. Mixing them creates unnecessary coupling.

Why TypedDict for AgentState instead of a regular dict?

TypedDict lets Python's type checker catch errors at edit time — mistyped field names warn you immediately. It also documents the entire data model in 18 lines. Anyone reading state.py instantly knows every piece of data in the system.

Why different temperatures for different nodes?

Guard (0) and Evaluator (0) make classification decisions — same input must always give same output. Query Builder (0.3) needs slight creativity to rephrase differently. Psychologist (0.7) and Follow-up (0.6) need warmth and natural language variation.

Why with_structured_output() instead of PydanticOutputParser?

PydanticOutputParser asked LLM to return raw JSON text and then parsed it. Groq returned normal prose and the parser crashed with JSONDecodeError. with_structured_output() uses Groq's tool-calling API — schema enforced at the API level, JSON parsing failures impossible.

Why is the same HuggingFace model used at ingest and query time?

Embeddings are positions in a 384-dimensional space. The same model always maps the same text to the same position. Different models = different spaces = cosine comparisons meaningless = random retrieval results.

Why 800 characters and not 1,000 (the default)?

Psychology books are concept-dense. A 1,000-char chunk might cover 3 attachment concepts. Its vector is pulled in 3 directions and matches none precisely. 800 chars captures one complete idea with a focused vector.

Why 120-char overlap?

Without overlap, a sentence starting at char 798 and ending at char 820 would be split across two chunks, appearing incomplete in both. 120-char overlap ensures boundary sentences always appear fully in at least one chunk.

Why was BM25 added then removed?

BM25 (keyword search) was added to complement cosine (semantic). After testing, response quality was equivalent with or without it (~8/10) because the evaluator+query_builder loop already compensates for weak initial retrieval. Removed for simplicity.

Why filter foreign book references in the retriever?

Some OceanofPDF PDFs are compilations with text from multiple books. A chunk labeled 'NVC' might contain 'The Gifts of Imperfection (p.23)'. Without filtering, the psychologist cites it as a real source — hallucinating a book not in the knowledge base.

Why does active_frameworks check response text?

Previously all retrieved books appeared in 'Sources' even if the response never cited them. The fix: only books whose name appears in the generated response text are shown as 'Used'.

Why local ChromaDB instead of a cloud vector database?

ChromaDB runs on disk — no external service, no API key, no cost, no rate limits, no network latency. For 7,267 chunks that rarely change, local is the right choice.

Why rebuild ChromaDB when changing books?

When a PDF is replaced, the old chunks with corrupted text still exist in ChromaDB. Rebuilding ensures ChromaDB exactly reflects the current books with clean text.

Every line. Every file. Every decision.

LangGraph · LangChain · Groq Llama 3.3 70B · ChromaDB
HuggingFace Sentence Transformers · FastAPI · React